StreamForge Explained Simply

Oct 26, 2026 · @KAr

What StreamForge is

StreamForge is a mini YouTube that runs on AWS. You upload a video, the system turns it into smaller versions, and then anyone can watch it in the browser.

It has 4 parts, and each one has a single job:

Part	Everyday comparison	Its job	Folder
1. Website	The shop counter	Lets you upload and watch videos	`services/frontend/`
2. API	The manager with a notebook	Keeps records and hands out upload links	`services/api/`
3. Worker	The kitchen	Converts raw videos into 720p, 480p and a thumbnail	`services/worker/`
4. Cloud setup and automation	The building and the delivery trucks	Builds AWS, tests and ships new code	`infra/`, `.github/`, `scripts/`

The big idea: no part waits on another. The website talks to the API, the video goes straight to storage, and the worker does the slow work in the background.

Part 1: The website (frontend)

The website is the only part a person sees. It is plain HTML, CSS and JavaScript, so there is no framework to learn. It is styled to look like YouTube.

Files

- `index.html` is the page layout: the video grid, the watch page and the upload pop-up.
- `style.css` handles the look: colours, cards and the dark theme.
- `app.js` holds all the logic: uploading, polling and playing.
- `config.js` tells the page where the API is. On your laptop that is `http://localhost:8000`. When deployed, the CI pipeline changes it to `/api`.

What you can do on it

1. **Upload a video.** Click upload, drag in an MP4 and type a title.
2. **Watch the progress bar.** The browser sends the file with `XMLHttpRequest`, which reports how much has been sent so far.
3. **Wait for "ready".** Every 5 seconds the page asks the API for the video list and redraws the cards.
4. **Watch it.** Click a card to open the watch page. Use the quality pills to switch between 720p and 480p.
5. **Search and filter.** Type in the search box to filter videos by title.

The extras are just for show. Likes, subscribe and comments only live in your own browser (`localStorage`). The server never sees them, and the view counts are made up from the video id.

One key idea: the website never sends the video through the API. It asks the API for a special upload link, then sends the file straight to S3. Part 2 explains why.

Part 2: The API (the manager)

The API is a small Python server built with FastAPI. It keeps a record of every video and gives out upload links. It never touches the video file itself.

Files

- `app/main.py` defines the web addresses (endpoints) the website calls.
- `app/aws.py` talks to AWS: it reads and writes the database and makes upload links.
- `app/models.py` describes what a video record looks like.
- `app/config.py` reads settings such as bucket and table names from environment variables.

The endpoints

Request	What it does
`POST /api/videos` with a title	Creates a new video record and returns its id plus an upload link
`GET /api/videos`	Lists all videos, newest first, with play links for the ready ones
`GET /api/videos/{id}`	Returns one video, or 404 if it doesn't exist
`GET /health`	Answers "healthy". The load balancer uses this to check the API is alive

The database (DynamoDB). Each video is one row in a table called `videos`. The row holds the title, a status, the file locations, the number of attempts and any error message. The status moves through these steps:

`uploaded` → `processing` → `ready`, or `failed` if something goes wrong.

The upload link (presigned URL). This is the clever part. Instead of receiving the video, the API asks S3 for a one-time link that allows exactly one thing: putting a file at `raw/{video_id}.mp4`. The link expires after 15 minutes (900 seconds). The browser then uploads straight to S3.

Why do it this way? Videos are big. If they went through the API, the server would run out of memory and get slow. With this trick the API only ever handles tiny text messages.

Play links. When a video is `ready`, the API adds full CloudFront links for the 720p file, the 480p file and the thumbnail. The website can play them directly.

Part 3: The worker (the video converter)

The worker is a Python program that turns a raw upload into videos that are ready to stream. It runs in the background and only starts up when there is work waiting.

Files

- `worker/main.py` holds the loop: get a job, process it, report the result.
- `worker/transcoder.py` builds and runs the FFmpeg commands.
- `worker/config.py` reads settings such as the queue address, bucket names and timeout.

How a job reaches the worker. When a file lands in the uploads bucket, S3 automatically drops a message into a queue (SQS) called `video-jobs`. Think of the queue as a to-do list. The worker asks it "any jobs?" and waits up to 20 seconds for an answer. This is called long polling, and it is cheaper than asking over and over.

What the worker does with each job

1. It checks the database. If the video is already `ready`, the message is a duplicate and gets skipped.
2. It sets the status to `processing` and adds 1 to the attempt count.
3. It downloads the raw MP4 into a temporary folder.
4. It runs FFmpeg 3 times:
   - **720p**: H.264 video, AAC audio
   - **480p**: same settings, smaller size

    - **Thumbnail**: one frame at 1 second, 640 px wide. Clips shorter than 1 second use the first frame instead.
5. It uploads the results to the processed bucket: `processed/{id}/720p.mp4`, `processed/{id}/480p.mp4` and `thumbnails/{id}/thumb.jpg`.
6. It sets the status to `ready` and saves where the files are.
7. It deletes the message from the queue, so the job counts as done.
8. It cleans up the temporary folder, whether the job worked or not.

The FFmpeg settings are `-preset veryfast -crf 28`. That means fast encoding at decent quality. `+faststart` lets a video start playing before it has fully downloaded.

When something breaks

- If FFmpeg fails, the worker marks the video `failed` and saves the error. It does **not** delete the message.
- After 5 minutes (the visibility timeout), SQS hands the message out again for another try.
- After 3 failed tries, SQS moves the message to a separate queue called `video-jobs-dlq` (the dead-letter queue). It stays there for 14 days so someone can look into it.
- This stops one broken video from being retried forever and running up the bill.
- A garbled message that can't be read is thrown away straight away.
- When AWS asks a worker to stop (SIGTERM), it finishes its current job first and then exits.

Scaling to zero (the money saver)

CloudWatch alarms watch the queue and change how many workers run:

- **Scale out:** if 1 or more jobs wait for 1 minute, add workers. 1 to 4 waiting jobs add 1 worker, 5 to 9 add 2, and 10 or more add 3. There are never more than the maximum (3 by default).
- **Scale in:** if the queue has no waiting jobs and no jobs in progress for 5 minutes, go down to 0 workers.

Counting in-progress jobs matters: it stops AWS from shutting down a worker halfway through a video.

Workers also run on **Fargate Spot**. That is spare AWS capacity sold at a big discount (around 70% off). AWS can take it back at short notice, but a job that gets cut off simply goes back to the queue and is retried.

Part 4: Cloud setup and automation

This part builds the AWS "building" that the other 3 parts live in, and ships new code there automatically. It has 4 pieces: Terraform, CloudFront, GitHub Actions and the local setup.

Terraform: AWS written as code

Terraform files describe every AWS resource. When you run `terraform apply`, AWS is built to match the files. Each file covers one area:

File	What it creates
`network.tf`	A private network (VPC) with public subnets. There is no NAT gateway, which saves about $32 a month
`s3.tf`	3 storage buckets: uploads (raw videos), processed (finished videos) and frontend (the website files)
`sqs.tf`	The `video-jobs` queue and its dead-letter queue
`dynamodb.tf`	The `videos` table
`ecr.tf`	Places to store the Docker images for the API and the worker
`ecs.tf`	The cluster that runs the containers. The API runs on normal Fargate, the worker on Fargate Spot
`autoscaling.tf`	The alarms that scale workers between 0 and 3
`cloudfront.tf`	The CDN (see below)
`iam.tf`	Permissions: who is allowed to do what, including the GitHub login role

The containers sit in public subnets, but security groups block all incoming traffic. The only exception is the load balancer reaching the API.

CloudFront: one front door for everything

CloudFront gives the whole app one HTTPS address and caches files in data centres around the world. It sends each request to the right place:

- `/api/*` goes to the load balancer, then the API.
- `/processed/*` and `/thumbnails/*` go to the processed bucket.
- Everything else goes to the frontend bucket, which holds the website.

Because everything shares one address, the browser has no cross-site (CORS) problems. The buckets also stay fully private. Only CloudFront can read them, through a feature called Origin Access Control (OAC).

GitHub Actions: automatic testing and deploys

There are 3 workflows in `.github/workflows/`:

`ci-deploy.yml` runs on every push to `main`:

1. **Find changes:** works out which parts changed, so only those get rebuilt.
2. **Test:** runs `ruff` (style checks) and `pytest` (tests) for the API and the worker.
3. **Build and scan:** builds the Docker image, then scans it with Trivy for security holes. The scan runs before any AWS login happens, so the scanner never sees AWS credentials.
4. **Deploy:** pushes the image to ECR and updates the ECS service.
5. **Check for rollback:** if the new containers keep crashing, the ECS circuit breaker switches back to the old version. The workflow then fails so you find out.
6. **Frontend:** sets `config.js` to `/api` and uploads the website files to S3.
7. **Smoke test:** calls `/health` through both the load balancer and CloudFront, and prints the app URL.

`infra.yml` runs Terraform checks (`fmt` and `plan`) on pull requests, so infrastructure changes get reviewed before they go live.

`rollback.yml` is a button you press by hand. Pick `api` or `worker`, and it goes back to the previous version (or one you choose).

To log in to AWS, the workflows use an IAM role through OIDC, a short-lived token from GitHub, so no long-lived keys are needed. Access keys stored in GitHub Secrets work as a backup.

Running locally and the helper scripts

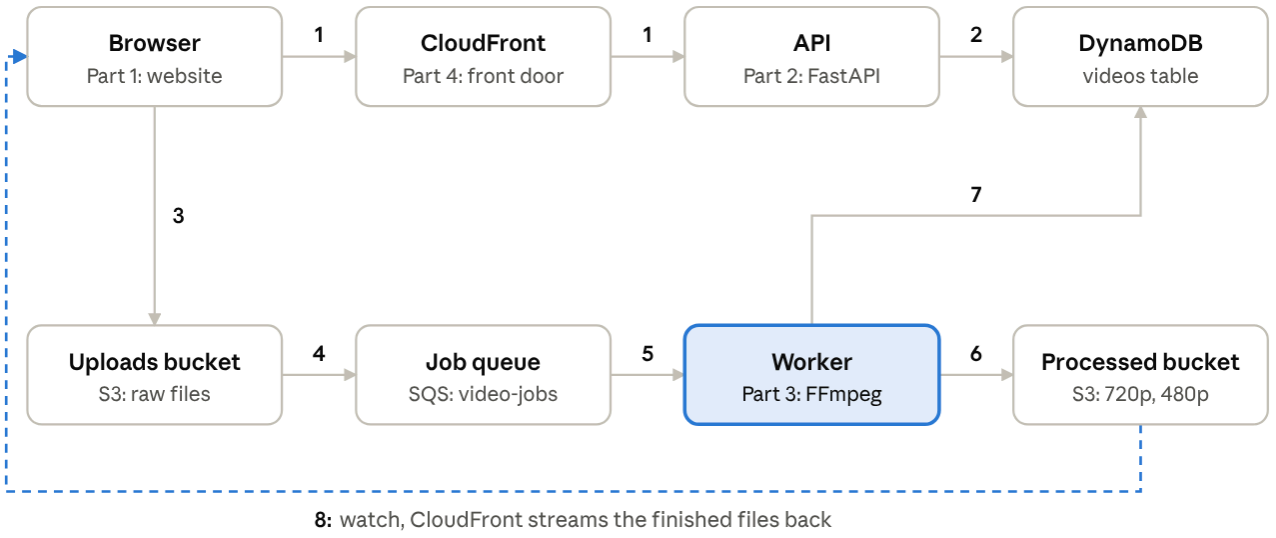
- **`docker-compose.yml`** runs the whole system on your laptop for free. LocalStack pretends to be S3, SQS and DynamoDB. Run `docker compose up --build` and open `http://localhost:3000`.
- **`scripts/init-localstack.sh`** creates the fake buckets, queue and table when LocalStack starts.
- **`scripts/bootstrap-aws.sh`** does the whole cloud setup in one command. It checks your tools, runs Terraform and sets up GitHub. It is safe to run again.
- **`scripts/teardown-aws.sh`** deletes everything Terraform made, so nothing keeps costing money. Only the small Terraform state bucket is kept.

The journey of one video

One upload passes through all 4 parts in 8 steps. Only steps 1 to 3 involve you. Everything after that happens in the background.

A video goes in through S3, the worker converts it, and it comes out through CloudFront

Numbers match the steps listed below the drawing



journey of one video · 8 steps across the 4 parts

1. **Ask.** You type a title and press upload. The browser sends `POST /api/videos` through CloudFront to the API.
2. **Record.** The API saves a row in DynamoDB with status `uploaded`, then returns the video id and a 15-minute upload link.
3. **Upload.** The browser sends the MP4 straight to the uploads bucket at `raw/{id}.mp4`, and the progress bar fills.
4. **Queue.** S3 notices the new file and adds a job message to `video-jobs`.
5. **Pick up.** Autoscaling starts a worker (from 0 if needed). The worker takes the job and sets the status to `processing`.
6. **Convert.** FFmpeg makes the 720p file, the 480p file and the thumbnail, and the worker uploads them to the processed bucket.
7. **Finish.** The worker sets the status to `ready` and deletes the job. After 5 quiet minutes, the workers scale back to 0.
8. **Watch.** The website's 5-second check sees `ready`, shows the card, and plays the video through CloudFront.

Cost and safety in one glance

Choice	What it saves or protects
Workers scale to 0	No compute cost while idle
Fargate Spot for workers	About 70% cheaper than normal Fargate
No NAT gateway	About $32 a month
Direct-to-S3 uploads	The API stays small and fast
Dead-letter queue after 3 tries	A broken video can't loop forever
Private buckets plus OAC	Files can only be read through CloudFront
Trivy scan before deploy	Images with known security holes get caught
Circuit breaker plus rollback workflow	A bad release goes back to the last good one

Glossary

Word	Plain meaning
AWS	Amazon's cloud: computers and services you rent
S3	File storage in the cloud, organised into "buckets"
SQS	A queue, like a to-do list that programs read from
DynamoDB	A fast database that stores one row per video
ECS / Fargate	A service that runs Docker containers without you managing servers
Fargate Spot	Spare Fargate capacity at a discount. AWS can take it back
ECR	Storage for Docker images
ALB	Application Load Balancer. It spreads requests across API containers
CloudFront	A CDN: copies of your files served from locations near users
OAC	Origin Access Control. It lets only CloudFront read the buckets
Presigned URL	A temporary link that allows one specific upload

Word	Plain meaning
FFmpeg	A free tool that converts video and audio
720p / 480p	Video heights in pixels: HD and standard quality
Terraform	A tool that builds cloud resources from code files
CI/CD	Automatic testing (CI) and automatic releasing (CD)
OIDC	A way for GitHub to log in to AWS with short-lived tokens
Trivy	A scanner that finds known security holes in images
LocalStack	A fake AWS that runs on your laptop
Dead-letter queue (DLQ)	Where failed jobs go after too many tries